

【全自動に超進化】100ページ超のPDF要件定義書も余裕！Antigravityだけで一気に設計書に落とし込む！

🔗 来源: <https://qiita.com/hitotch/items/Oe746edf66e123a7af81>

Antigravity「だけ」で大規模PDF仕様書から大規模Wordドキュメントを作成する最新ワークフロー

大規模な要件定義書や仕様書（数百ページのPDFなど）をAIに読み込ませ、そこからシステム設計書や提案書などの新規ドキュメントを自動生成させたい。

以前の記事では「NotebookLM」などを使い、AIが処理できるサイズにPDFを分割して読み込ませる必要がありました。しかし、さらなる工夫によりその分割作業すら不要になり、全自動化に成功しました！

本記事では、AIネイティブIDE「Antigravity」と最新LLMの広大なコンテキストウィンドウを活かし、巨大なPDFを一切分割せずに一気に通貫で**Markdown**として再構築し、最終的に**Word**ファイル（.docx）を出力するまでの実践的な爆速ワークフローを解説します。

💡 筆者の環境

本記事のワークフローは、超広大なコンテキストウィンドウと高度なマルチモーダル能力を持つ **Gemini 3.1 Pro** を使用して検証しています。

💡 文書ではなくプレゼン資料を作りたい場合は以下の記事へ。

実務レベルのパワポを1分で！[Googleスライド/パワーポイント両対応のプレゼン自動生成エンジン「AI to Slides」](#)

ゴール

- 数百ページ規模のPDF仕様書を分割することなく、一気にMarkdown形式でドキュメント化する。
- Antigravity内でプロジェクトとして統合管理し、AIエージェントにドキュメントの生成・修正を行わせる。

- ドキュメント内の図表は「元の画像」として残しつつ、後からテキストベースで編集できるよう「Mermaid化」したコードも併記させる。
- 作成したMarkdown（Mermaid図含む）を、Antigravity上からWord形式へ一発変換する。

前提

- Antigravityの基本的な操作ができる。
- **Gemini 3.1 Pro**等の高性能モデルの利用（推奨）：数十～100ページ超のPDFを分割なしで実務レベルで処理する場合、広大なコンテキストウィンドウと強力なビジョン（視覚）能力を持つ最新モデルの利用を推奨します。

今回のお題となるサンプルPDFとして、デジタル庁が公開している「[調達仕様書等の概要（2026年3月6日版）](#)」を使用します。（出典：デジタル庁ウェブサイト）

用語集

本記事で出てくる、ドキュメント管理の核となる2つの技術について簡単に触れておきます。

- **Markdown**（マークダウン）
 - 特徴: プレーンテキストで手軽に見出しや箇条書きなどの構造を持たせることができる記述言語。
 - 必要な理由: AIエージェントにとって最も読み書きがしやすく、プロジェクト内の複数ドキュメントを横断して理解・修正させるのに最適なフォーマットだからです。
- **Mermaid**（マーメイド）
 - 特徴: テキストでコードを書くように、フローチャートやシーケンス図などの図表を生成できる記法・ツール。
 - 必要な理由: 図を「画像」ではなく「テキスト（コード）」として扱えるため、AIに「この処理フローを図に追加して」と自然言語で指示するだけで図面の修正が完了し、保守性が飛躍的に高まるためです。

0. Antigravityと必須プラグインのインストール

まずはベースとなるIDE環境を整えます。

1. Antigravityのインストール

[Antigravityの公式サイト](#) から、お使いのOSに合ったインストーラーをダウンロードし、インストールを完了させてください。VS CodeベースのUIとなっているため、普段エディタを触っている方なら迷わず導入できます。

2. 必須プラグインの導入

Antigravityを起動し、左側のメニューから「拡張機能 (Extensions)」アイコンを開き、今回のワークフローで必要になる以下の2つのプラグインを検索してインストールします。

- **Markdown Preview Mermaid Support (作者: bierner)**

Antigravity標準のMarkdownプレビュー画面で、Mermaid形式のコードを美しい図表として直接レンダリングするために使用します。

- **Markdown Preview Enhanced (作者: shd101wyy)**

完成したMarkdownファイル（および生成されたMermaidの図表）を、一発でWordファイル (.docx) にエクスポートするために使用します。

💡 WindowsではNVMとNodeを入れておこう

WindowsではこのあとのPDF→画像変換で環境構築に躓くことが多いです。事前にNVM (Node Version Manager) を使ってNode.jsをインストールしておきましょう。

PowerShellを開き、以下のコマンドを順番に実行して環境を構築します。

```
# 1. NVM for Windowsをインストール (*パッケージ名に注意! )
winget install CoreyButler.NVMforWindows

# --- ここで一度PowerShellを閉じて、再起動してください ---

# 2. Node.jsの安定版 (LTS) をインストールして有効化
nvm install lts
nvm use lts

# 3. npm実行時のセキュリティエラー (スクリプト実行不可) を防ぐためポリシーを変更
# (警告が出たら「Y」を入力してEnter)
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

最後に `node -v` と `npm -v` を実行し、両方のバージョン番号が表示されれば準備完了です！

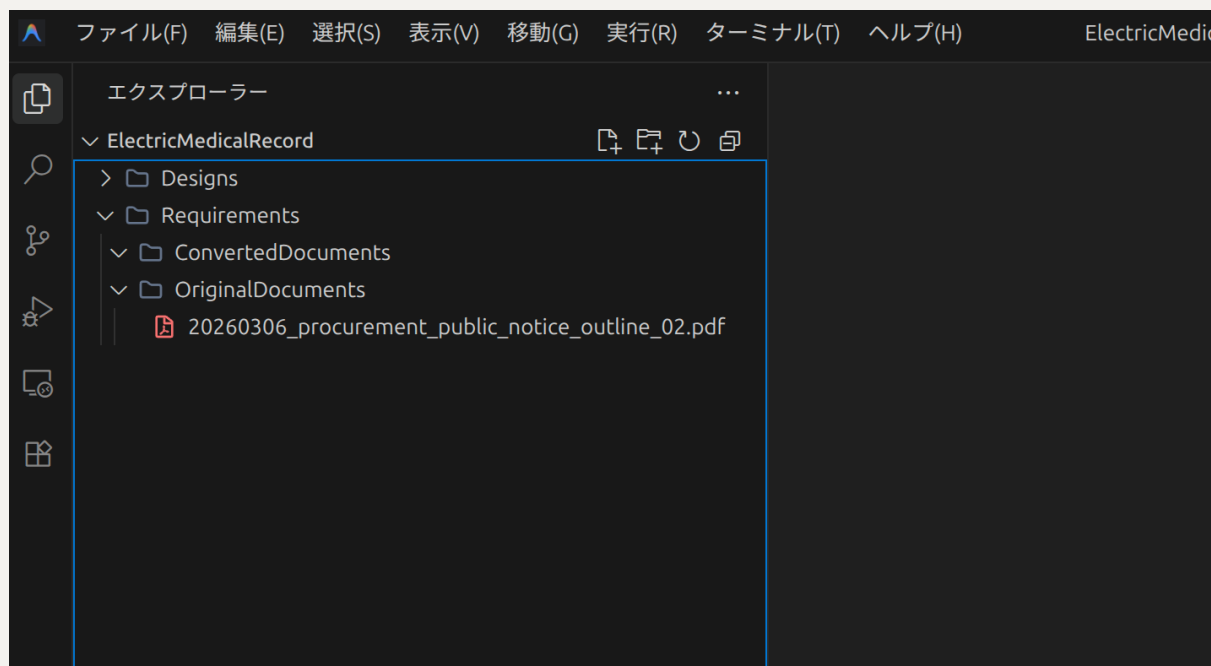
1. プロジェクトフォルダの作成とPDFの配置

Antigravity上でプロジェクトを作成し、対象となるPDFを所定のフォルダに配置します。今回は例として **ElectricMedicalRecord**（電子カルテ）というプロジェクトを想定します。

今回のお題となるサンプルPDFとして、デジタル庁が公開している「[調達仕様書等の概要（2026年3月6日版）](#)」を使用します。（出典：デジタル庁ウェブサイト）

Antigravity上で、以下のようなディレクトリ構成を作成してください。

```
ElectricMedicalRecord
├── Requirements
│   ├── OriginalDocuments ← ここに原本のPDF
│   │   │   (20260306_procurement_public_notice_outline_02.pdf) を入れる
│   └── ConvertedDocuments ← ここにMarkdownが出力される
└── Designs
```



2. AntigravityのAIに一括変換を指示する（プロンプトが鍵！）

対象のPDFを配置したら、Antigravityのチャットから一発でプロンプトを投げるだけです。

ここで完成品の品質を最大化するために、非常に重要なプロンプトの工夫があります。それは**「PDFを一度画像として読み込ませてからMarkdown化する」とことと、「図表の画像とMermaidを両立させる」**ことです。

以下のプロンプトをコピーして、AntigravityのAI（Agent）に指示を出してください。

実行するプロンプト

Requirements/OriginalDocuments/20260306_procurement_public_notice_outline_02.pdf

このファイルを読み込んで、Requirements/ConvertedDocuments フォルダ内に同じ内容のマークダウンファイルを作ってください。

【完成品の品質を最大化するための重要手順】

1. テキストを単に抽出するのではなく、一度PDFの各ページを「画像」に落とし込み、視覚的なレイアウトや図解の文脈を深く理解してからマークダウンを作成してください。
2. 文中に図や複雑な表がある場合、そのまま画像としてマークダウン内に残してください。
3. さらに、可能であればその図を「mermaid」に変換した図も、画像の直後に付けてください。

ElectricMedicalRecord

 20260306_procurement_public_notice_outline_02.pdf

このファイルを読み込んで、 ConvertedDocuments フォルダ内に同じ内容のマークダウンファイルを作ってください。

【完成品の品質を最大化するための重要手順】

1. テキストを単に抽出するのではなく、一度PDFの各ページを「画像」に落とし込み、視覚的なレイアウトや図解の文脈を深く理解してからマークダウンを作成してください。
2. 文中に図や表がある場合、そのまま画像としてマークダウン内に残してください。
3. さらに、可能であればその図を「mermaid」に変換した図も、画像の直後に付けてください。

+ ∨ Planning ∨ Gemini 3.1 Pro (High)



Windowsでは画像変換にローカルの**Node**を使うように指示しよう

Windows環境では、AIがPythonなどの別のツールを使ってPDFを画像変換しようとする、ライブラリの依存関係などでエラーになることがよくあります。

そのため、上記でせっかく構築したNode環境を確実に使わせるために、プロンプトの最後に以下の1文を付け足すと成功率がグッと上がります。

4. 画像変換処理を行う場合は、ローカル環境にインストール済みのNode.jsを使用してください。

なぜこの指示が必要なのか？

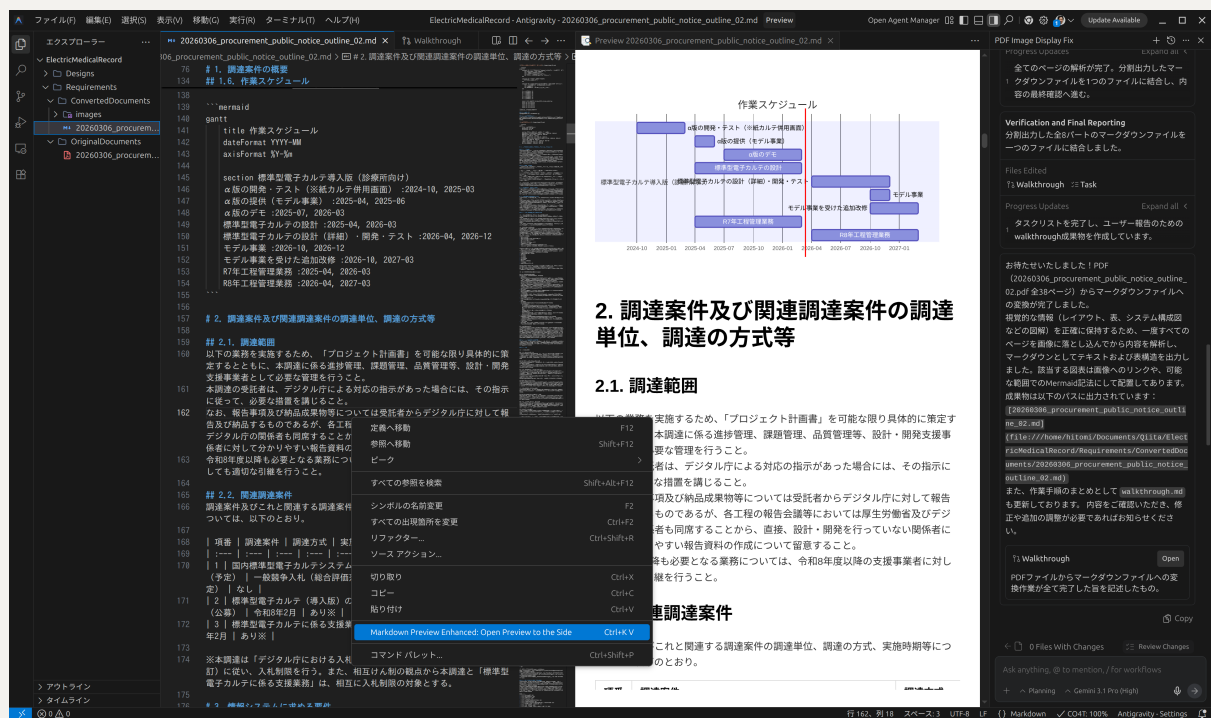
複雑な仕様書は、テキスト情報だけの抽出では段落の意図や図解との紐づきが破綻しがちです。最新モデルの強力なビジョン（視覚）能力を使って「人間が目で見えるように」全体を把握させることで、文書構造を正確に捉えた美しいMarkdownが生成されます。

さらに、元の図表を画像として残すことで「原本の正確なニュアンス」を担保しつつ、直後にMermaidコードを出力させることで「後からフローを修正・追加できる保守性」を同時に獲得できます。

3. 出力結果の確認とドキュメントの編集

プロンプトを実行すると、Antigravityが自動的にPDFを解析し、文書全体の見出し構造を維持したまま整理されたマークダウンファイルが生成されます。

プレビューしたい場合は右クリックメニューからプレビューを開くことができます。



ここからがAntigravityの真骨頂です。生成されたディレクトリをベースに、チャット欄から以下のように自然言語で指示を出します。

この調達仕様書のディレクトリをベースに、Designsフォルダ内にまずはシステム構成案のドキュメントを作って。

- ・AWSの場合とGCPの場合の2つを考えて。
- ・構成図はmermaidで作って。

The screenshot shows the Antigravity application interface. On the left is a file explorer showing a project structure with folders like 'Designs' and 'SystemArchitecture'. The main editor displays a Mermaid diagram titled 'AWS構成 概念図 (Mermaid Graph)'. The diagram illustrates an AWS-based system architecture for a medical record system. It includes components like CloudFront, S3, ECS, ALB, Amazon WAF, Amazon Aurora, and Amazon RDS. The diagram is titled 'AWS構成 概念図 (Mermaid Graph)'. On the right, there's a sidebar with a preview of the diagram and a notes section. The notes mention that the design is based on the 'Designs' folder and that the system architecture is being designed using Mermaid.

※実践では構成図にはmermaidではなく見た目がきれいになるdraw.ioを指定してもよいでしょう。Antigravityでは拡張機能でdraw.ioも扱えます。

Agentがプロジェクト全体を俯瞰し、矛盾のないドキュメントを自動生成・修正してくれます。人間は内容の精査とAgentへの指示出しに専念します。

図の修正が必要な場合も、Agentに「このMermaid図の認証シーケンスに、マイナンバーカードを用いたプロセスを追加して」と指示するだけで、コードを正確に書き換えてくれます。

4. Wordファイル (.docx) へのエクスポート環境を作る

最終的な提出物がWord形式で求められるケースに対応するため、Antigravity内でMarkdownからWordへ一発変換する環境を構築します。

必要なツールのインストール

図の生成に必要な `Node.js (npx)` と、Word変換に必要な `Pandoc` をOSにインストールします。

- **Windows**の場合:

現在のWindowsでは、PowerShellから標準パッケージマネージャーの `winget` を使用するのが最も簡単です。

※手順0でNVMを使ってすでにNode.jsはインストール済みのため、ここではPandocのみを追加します。

```
# Pandocのインストール
```

```
winget install JohnMacFarlane.Pandoc
```

(※過去に `winget install OpenJS.NodeJS` などNode.jsを直接入れてしまっている場合は、NVMと競合してエラーの原因になるため、アンインストールしてNVMでの管理に統一することをおすすめします。)

- **Mac**の場合:

Homebrewを使ってインストールするのが標準的です。

```
brew install node
```

```
brew install pandoc
```

- **Ubuntu / WSL**の場合:

`apt` などのパッケージマネージャーを使用してインストールします。

```
sudo apt update
```

```
sudo apt install nodejs npm pandoc
```

Antigravityのセットアップ

1. Antigravityの拡張機能アイコンから `Markdown Preview Enhanced` (作者: `shd101wyy`) をインストール済みであることを確認します。
2. 【重要】 新しくインストールした `npx` や `pandoc` のコマンドパスをIDEに認識させるため、**Antigravity**のウィンドウを一度完全に閉じて、再起動 してください。

Markdownファイルの準備

出力フォーマットエラーを防ぐため、対象のMarkdownファイルの**先頭（1行目）**に以下のフロントマターを追記します。


```
---  
output: word_document  
---
```

エクスポートの実行

1. 対象のファイルを開き、`Markdown Preview Enhanced` のプレビュー画面を表示します。
2. プレビュー画面上で 右クリック > 「**Pandoc**」 > 「**docx**」 を選択します。

バックグラウンドで `npx` がMermaidを画像に変換し、同ディレクトリにWordファイル（.docx）が出力されます。

以上で、巨大な要件定義書を一括処理し、設計を経てWordドキュメントを生成する一連のワークフローは完了です。ぜひ皆さんのプロジェクトでも試してみてください！

0

[コメント一覧へ移動](#)

[X \(Twitter\) でシェアする](#)

[Facebookでシェアする](#)

[はてなブックマークに追加する](#)